

Your grade: 80%

Your latest: 80% • Your highest: 80% • To pass you need at least 80%. We keep your highest score.

Next item →

1. What is the key advantage of having CNNs learn their own filters rather than using hand-designed filters?

1 / 1 point

- ☐ Learned filters work for any image classification task without retraining
- ☐ Hand-designed filters are too slow to compute
- ☒ The model discovers task-specific patterns automatically
- ☐ Hand-designed filters need extra storage for configuration metadata

✔ Nice work

Yes. Instead of manually determining which filters work best for butterflies versus flowers versus beetles, CNNs automatically tune their filters to find the specific patterns that identify each species. This is what makes them powerful for specialized classification tasks.

2. What is the sequence of operations that happens when a filter performs a convolution at one position in an image?

1 / 1 point

- ☐ Add filter weights to pixel values, then multiply the results together
- ☐ Select the largest pixel value in the grid and discard the others.
- ☒ Multiply the filter values with the pixel values, sum the products, and take the average.
- ☐ Average the pixel values first, then multiply by the filter.

✔ Nice work

Yes. As you slide the filter (a grid of numbers) over the image, you multiply the filter values by the pixel values underneath one by one. Then, you add those products together and take the average to calculate the new pixel value.

3. A CNN has two convolutional layers, each followed by MaxPool2d with kernel_size=2. If the input image is 28×28 pixels, what size are the feature maps after both pooling layers?

1 / 1 point

- ☐ 28×28
- ☐ 14×14
- ☒ 7×7
- ☐ 3×3

✔ Nice work

Yes. The first max-pooling layer halves 28×28 to 14×14, and the second max-pooling layer halves that to 7×7. Each pooling layer with kernel_size=2 reduces each spatial dimension by half.

4. In a convolutional layer with in_channels=3 and out_channels=32, what does the 32 represent?

1 / 1 point

- ☐ The layer processes 32 images at once in a batch
- ☐ The layer creates 32 copies of each input channel
- ☒ The layer learns 32 different filters
- ☐ The output image will be 32×32 pixels

✔ Nice work

Yes. With out_channels=32, the model learns 32 different filters. Each filter picks up different patterns from the 3 input channels (red, green, blue). This produces 32 feature maps showing where different patterns appear in the image.

5. What is a key limitation of using nn.Sequential to define a model in PyTorch?

1 / 1 point

- ☐ Sequential models cannot use convolutional layers
- ☐ Sequential models train slower than models using nn.Module
- ☒ Sequential doesn't support conditionals, loops, or branching
- ☐ Sequential cannot be used with the Adam optimizer

✔ Nice work

Yes. Sequential runs data through each layer in order with no flexibility. You can't add if statements to

handle different inputs differently, can't use loops, and can't print intermediate values to debug. Everything follows the same fixed path, which is why it feels like static graph programming even though PyTorch supports dynamic graphs.

6. What is the core difference between how static graphs and PyTorch's dynamic graphs work?

1 / 1 point

- ☐ Static graphs run faster but dynamic graphs use less memory
- ☒ Static graphs are defined once before training, while dynamic graphs are built fresh each time data flows through
- ☐ Static graphs require defining everything in one block while dynamic graphs split definitions across methods
- ☐ Static graphs only work with images while dynamic graphs work with any data type

✔ Nice work

Yes. In static frameworks, you define the entire computation graph up front before anything runs, and the structure is locked. PyTorch builds a new graph each time the forward method runs, recording exactly what happens based on the actual path your data takes. This graph is used for backpropagation, then discarded—ready to build a brand new one next time.

7. Why does PyTorch separate the `__init__` and forward methods instead of combining them into a single method?

1 / 1 point

- ☐ It's a Python convention that all classes must follow
- ☐ The `__init__` method runs faster than the forward method
- ☒ `__init__` defines the architecture, forward defines data flow
- ☐ It makes the code easier to read by splitting it into smaller sections

✔ Nice work

Yes. This separation of concerns is fundamental to PyTorch's design. The `__init__` method defines your model's architecture and learnable parameters that persist across training, while the forward method defines how data flows through the model, enabling PyTorch's dynamic graph flexibility.

8. When defining a custom `nn.Module`, what is one benefit of using `nn.Sequential` to organize its layers?

0 / 1 point

- ☐ Sequential makes the model train faster
- ☒ Sequential allows you to use conditionals and loops in your model
- ☐ You don't need to manually call each layer in forward
- ☐ Sequential automatically chooses the best layer order for your data

✘ Try again

No. Actually, Sequential does the opposite—it forces a fixed, linear path. For conditionals and loops, you need to define layers individually in forward.

9. What is the standard way to count the total number of parameters in a PyTorch model?

1 / 1 point

- ☐ `len(model.parameters())`
- ☐ `model.count_parameters()`
- ☒ `sum(p.numel() for p in model.parameters())`
- ☐ `print(model.parameters())`

✔ Nice work

Yes. The `.numel()` method returns the number of elements in each tensor. By iterating through all parameters and summing their element counts, you get the total parameter count. This is the standard PyTorch pattern for counting parameters.

10. What is the key difference between `model.children()` and `model.modules()` when inspecting a PyTorch model?

0 / 1 point

- ☒ `children()` shows parameters while `modules()` shows layers
- ☐ `children()` is faster but `modules()` is more detailed
- ☐ `children()` shows top-level components, `modules()` shows everything
- ☐ `modules()` only works with Sequential blocks while `children()` works with all layers

✘ Try again

No. Both methods show layers, not parameters. The difference is in how deeply they inspect nested structures like Sequential blocks.